

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL 8
QUEUE**



Disusun Oleh :

NAMA : RAIHAN DZAKY MUFLIH

NIM : 103112430029

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Antrean (*Queue*) adalah kumpulan data yang mengikuti aturan masuk dan keluar seperti antrean di kehidupan nyata, yaitu Siapa yang datang duluan, dia yang dilayani duluan (First-In, First-Out atau FIFO). Antrean memiliki dua ujung utama: Head (kepala), tempat data dikeluarkan, dan Tail (ekor), tempat data baru dimasukkan. Ada tiga cara utama mengimplementasikan antrean menggunakan wadah (*array*): Head Diam (membutuhkan pergeseran data saat keluar), Head Bergerak Linear (cepat, tetapi menyisakan ruang kosong di depan), dan Antrean Melingkar (*Circular Queue*) (paling efisien karena Tail dan Head berputar sehingga ruang kosong bisa dipakai lagi).

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

#queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX_QUEUE 5

struct Queue
{
    int info[MAX_QUEUE];
    int head;
    int tail;
    int count;
};

void createQueue(Queue &Q);

bool isEmpty(Queue Q);

bool isFull(Queue Q);

void enqueue(Queue &Q, int x);

int dequeue(Queue &Q);

void printInfo(Queue Q);

#endif
```

#queue.cpp

```
#include "queue.h"
#include <iostream>

using namespace std;

void createQueue(Queue &Q) {
    Q.head = 0;
    Q.tail = -1;
    Q.count = 0;
}

bool isEmpty(Queue Q) {
    return Q.count == 0;
}

bool isFull(Queue Q) {
    return Q.count == MAX_QUEUE;
}

void enqueue(Queue &Q, int x) {
    if (!isFull(Q)) {
        Q.tail = (Q.tail + 1) % MAX_QUEUE;
        Q.info[Q.tail] = x;
        Q.count++;
    } else {
        cout << "Antrean Penuh! " << endl;
    }
}

int dequeue(Queue &Q) {
    if (!isEmpty(Q)) {
        int x = Q.info[Q.head];
        Q.head = (Q.head + 1) % MAX_QUEUE;
        Q.count--;
        return x;
    } else {
        cout << "Antrean Kosong! " << endl;
        return -1;
    }
}
```

```

}

void printInfo(Queue Q) {
    if (isEmpty(Q)) {
        cout << "Isi Antrean: [Kosong]" << endl;
        return;
    }

    cout << "Isi Antrean: [";

    int i = Q.head;
    for (int n = 0; n < Q.count; n++) {
        cout << Q.info[i];
        if (n < Q.count - 1) cout << ", ";
        i = (i + 1) % MAX_QUEUE;
    }

    cout << "]" << endl;
}

```

#main.cpp

```

#include "queue.h"
#include "queue.cpp"
#include <iostream>

using namespace std;

int main() {
    Queue Q;

    createQueue(Q);
    printInfo(Q);

    cout << "\n enqueue 3 elemen" << endl;
    enqueue(Q, 5);
    printInfo(Q);
    enqueue(Q, 2);
    printInfo(Q);
}

```

```

    enqueue(Q, 7);
    printInfo(Q);

    cout << "\n Dequeue 1 elemen"<<endl;

    cout<< "Elemen keluar: " << dequeue(Q) << endl;
    printInfo(Q);

    cout << "\n enqueue 1 elemen"<<endl;
    enqueue(Q, 4);
    printInfo(Q);

    cout << "\n Dequeue 2 elemen"<<endl;
    cout<< "Elemen keluar: " << dequeue(Q) << endl;
    cout << "Elemen keluar: " << dequeue(Q) << endl;
    printInfo(Q);

    return 0;
}

```

Screenshots Output

Isi Antrean: [Kosong]

enqueue 3 elemen

Isi Antrean: [5]

Isi Antrean: [5, 2]

Isi Antrean: [5, 2, 7]

Dequeue 1 elemen

Elemen keluar: 5

Isi Antrean: [2, 7]

enqueue 1 elemen

Isi Antrean: [2, 7, 4]

Dequeue 2 elemen

Elemen keluar: 2

Elemen keluar: 7

Isi Antrean: [4]

PS D:\C++\MODUL 8>

Deskripsi:

Program ini menunjukkan cara membuat antrian melingkar / Circular Queue menggunakan array dengan bantuan penghitung jumlah barang. Kita menggunakan penghitung ini untuk tahu persis berapa banyak barang yang sudah masuk, sehingga kita bisa memastikan antrean benar-benar kosong atau benar-benar penuh. Ketika barang dimasukkan atau dikeluarkan, posisi kepala dan ekor akan bergerak memutar dengan menggunakan hitungan sisa bagi. Pergerakan memutar ini membuat tempat yang ditinggalkan di awal antrean bisa dipakai lagi, sehingga penggunaan tempat di array menjadi lebih efisien.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

#Header queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#include <iostream>
#include <cstdlib>

typedef int infotype;
#define MAX_SIZE 5

typedef struct {
    infotype A[MAX_SIZE];
    int head;
    int tail;
} Queue;

void CreateQueue(Queue &Q);
bool isEmptyQueue(Queue Q);
void printInfo(Queue Q);
bool isFullQueue(Queue Q);
void enqueue(Queue &Q, infotype x);
infotype dequeue(Queue &Q);

#endif
```

1. Alternatif 1

#queue.cpp

```
#include "queue.h"

void CreateQueue(Queue &Q) {
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue(Queue Q) {
    return (Q.head == -1 && Q.tail == -1);
}

bool isFullQueue(Queue Q) {
    return (Q.tail == MAX_SIZE - 1);
}

void printInfo(Queue Q) {
    std::cout << Q.head << " \t " << Q.tail << " \t : ";

    if (isEmptyQueue(Q)) {
        std::cout << "empty queue" << std::endl;
    } else {
        for (int i = Q.head; i <= Q.tail; i++) {
            std::cout << Q.A[i];
            if (i < Q.tail) {
                std::cout << " ";
            }
        }
        std::cout << std::endl;
    }
}

void enqueue(Queue &Q, infotype x) {
    if (isFullQueue(Q)) {
        return;
    }
    if (isEmptyQueue(Q)) {
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail++;
    }
    Q.A[Q.tail] = x;
}
```

```

}

infotype dequeue(Queue &Q) {
    if (isEmptyQueue(Q)) {
        return 0;
    }
    infotype removed_value = Q.A[Q.head];
    if (Q.head == Q.tail) {
        CreateQueue(Q);
    } else {
        for (int i = Q.head; i < Q.tail; i++) {
            Q.A[i] = Q.A[i + 1];
        }
        Q.tail--;
    }
    return removed_value;
}

```

#main.cpp

```

#include "queue.h"
#include "queue.cpp"

void run_test() {
    std::cout << "Hello World" << std::endl;
    std::cout << "H \t T \t : Queue Info" << std::endl;
    std::cout << "-----" <<
std::endl;

    Queue Q;
    CreateQueue(Q);
    printInfo(Q);
    enqueue(Q, 5); printInfo(Q);
    enqueue(Q, 2); printInfo(Q);
    enqueue(Q, 7); printInfo(Q);

    dequeue(Q); printInfo(Q);
    enqueue(Q, 4); printInfo(Q);
    dequeue(Q); printInfo(Q);
    dequeue(Q); printInfo(Q);
    dequeue(Q); printInfo(Q);
}

```



```

        printInfo(Q);
    }

int main() {
    run_test();
    return 0;
}

```

Screenshots Output

```

Hello World
H      T      : Queue Info
-----
-1     -1     : empty queue
0       0      : 5
0       1      : 5 2
0       2      : 5 2 7
0       1      : 2 7
0       2      : 2 7 4
0       1      : 7 4
0       0      : 4
-1     -1     : empty queue
-1     -1     : empty queue
PS D:\C++\MODUL 8\UNGUIDED>

```

Deskripsi:

Program Alternatif 1 adalah cara membuat antrian di mana posisi kepala (Head) selalu diam di tempat paling awal (indeks 0) sementara ekor (Tail) bergerak maju saat ada barang masuk. Ketika barang dikeluarkan, semua barang yang tersisa di antrian harus digeser maju satu per satu untuk menempati posisi yang ditinggalkan oleh kepala.

2. Alternatif 2

#queue.cpp

```

#include "queue.h"
#include <iomanip>

void CreateQueue(Queue &Q) { Q.head = -1; Q.tail = -1; }

```

```

bool isEmptyQueue(Queue Q) { return (Q.head == -1 && Q.tail ==
-1); }
bool isFullQueue(Queue Q) { return (Q.tail == MAX_SIZE - 1); }

void printInfo(Queue Q) {
    std::cout << std::setw(4) << Q.head;
    std::cout << std::setw(4) << Q.tail;
    std::cout << " : ";
    if (isEmptyQueue(Q)) {
        std::cout << "empty queue" << std::endl;
    } else {
        for (int i = Q.head; i <= Q.tail; i++) {
            std::cout << Q.A[i];
            if (i < Q.tail) {
                std::cout << " ";
            }
        }
        std::cout << std::endl;
    }
}

void enqueue(Queue &Q, infotype x) {
    if (isFullQueue(Q)) return;
    if (isEmptyQueue(Q)) {
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail++;
    }
    Q.A[Q.tail] = x;
}

infotype dequeue(Queue &Q) {
    if (isEmptyQueue(Q)) return 0;
    infotype removed_value = Q.A[Q.head];
    if (Q.head == Q.tail) {
        CreateQueue(Q);
    } else {
        Q.head++;
    }
    return removed_value;
}

```

#main.cpp

```
#include "queue.h"
#include "queue.cpp"
#include <iomanip>

void run_test_alt2() {
    std::cout << "Hello world!" << std::endl;
    std::cout << "H      T      : Queue Info" << std::endl;
    std::cout << "-----" <<
std::endl;

    Queue Q;
    CreateQueue(Q);
    printInfo(Q);
    for (int i = 1; i <= MAX_SIZE; i++) {
        enqueue(Q, i * 10);
        printInfo(Q);
    }
    dequeue(Q); printInfo(Q);
    dequeue(Q); printInfo(Q);
    enqueue(Q, 99);
    printInfo(Q);
}

int main() {
    run_test_alt2();
    return 0;
}
```

Screenshots Output

```
Hello world!
H      T      : Queue Info
-----
-1 -1 : empty queue
0  0 : 10
0  1 : 10 20
0  2 : 10 20 30
0  3 : 10 20 30 40
0  4 : 10 20 30 40 50
1  4 : 20 30 40 50
2  4 : 30 40 50
2  4 : 30 40 50
PS D:\C++\MODUL 8\UNGUIDED> 
```

Deskripsi:

Program Alternatif 2 adalah cara standar membuat antrian di mana kepala (Head) dan ekor (Tail) sama-sama bergerak maju ke kanan saat barang masuk dan keluar. Metode ini sangat cepat karena barang keluar hanya perlu menggeser posisi kepala ke kanan, tanpa perlu menggeser semua barang lain.

3. Alternatif 3

#queue.cpp

```
#include "queue.h"
#include <iomanip>

void CreateQueue(Queue &Q) {
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue(Queue Q) {
    return (Q.head == -1 && Q.tail == -1);
}

bool isFullQueue(Queue Q) {
    return ((Q.tail + 1) % MAX_SIZE == Q.head);
}

void printInfo(Queue Q) {
    std::cout << std::setw(4) << Q.head;
    std::cout << std::setw(4) << Q.tail;
    std::cout << " : ";

    if (isEmptyQueue(Q)) {
        std::cout << "empty queue" << std::endl;
    } else {
        int i = Q.head;
        do {
            std::cout << Q.A[i];
            if (i != Q.tail) {
                std::cout << " ";
            }
            i = (i + 1) % MAX_SIZE;
        } while (i != (Q.tail + 1) % MAX_SIZE);

        std::cout << std::endl;
    }
}
```

```

void enqueue(Queue &Q, infotype x) {
    if (isFullQueue(Q)) return;
    if (isEmptyQueue(Q)) {
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail = (Q.tail + 1) % MAX_SIZE;
    }
    Q.A[Q.tail] = x;
}

infotype dequeue(Queue &Q) {
    if (isEmptyQueue(Q)) return 0;
    infotype removed_value = Q.A[Q.head];

    if (Q.head == Q.tail) {
        CreateQueue(Q);
    } else {
        Q.head = (Q.head + 1) % MAX_SIZE;
    }
    return removed_value;
}

```

#main.cpp

```

// File: Alt3/main_alt3.cpp
#include "queue.h"
#include "queue.cpp"
#include <iomanip>

void run_test_alt3() {
    std::cout << "Hello world!" << std::endl;
    std::cout << "H      T      : Queue Info" << std::endl;
    std::cout << "-----" <<
std::endl;

    Queue Q;
    CreateQueue(Q);
    printInfo(Q);

    for (int i = 1; i < MAX_SIZE; i++) {
        enqueue(Q, i * 10);
        printInfo(Q);
    }
}

```

```

    }
    dequeue(Q); printInfo(Q);
    dequeue(Q); printInfo(Q);
    enqueue(Q, 99);
    printInfo(Q);
    enqueue(Q, 88);
    printInfo(Q);
    enqueue(Q, 77);
    printInfo(Q);
}

int main() {
    run_test_alt3();
    return 0;
}

```

Screenshots Output

```

Hello world!
H   T   : Queue Info
-----
-1  -1 : empty queue
0   0  : 10
0   1  : 10 20
0   2  : 10 20 30
0   3  : 10 20 30 40
1   3  : 20 30 40
2   3  : 30 40
2   4  : 30 40 99
2   0  : 30 40 99 88
2   1  : 30 40 99 88 77
PS D:\C++\MODUL 8\UNGUIDED>

```

Deskripsi:

Program Alternatif 3 menerapkan prinsip kepala dan ekor bergerak maju secara melingkar menggunakan perhitungan sisa bagi. Ketika ekor mencapai ujung akhir wadah, ia akan kembali berputar ke awal jika ada tempat kosong. Ini memungkinkan tempat yang ditinggalkan oleh kepala yang sudah keluar untuk digunakan kembali oleh ekor.

D. Kesimpulan

Tiga mekanisme utama diterapkan dalam mengimplementasikan Antrian (Queue) menggunakan array, dan masing-masing memiliki konsekuensi berbeda: Mekanisme Head Diam mudah dipahami, tetapi kurang efisien karena proses menggeser semua barang saat ada yang keluar memakan banyak waktu dan tenaga komputer, terutama jika antriannya panjang. Sementara itu, mekanisme Linear Array (Head Bergerak) menimbulkan masalah pemborosan tempat karena setelah barang keluar, tempat kosong di bagian awal array tidak bisa dipakai lagi, sehingga kita bisa kehabisan tempat meskipun array belum terisi penuh. Untuk mengatasi kelemahan ini, mekanisme Antrian Melingkar (Circular Queue) adalah yang paling optimal, di mana tidak ada tempat yang terbuang sia-sia, membuat wadah array terpakai secara maksimal.

E. Referensi

[Adam Drozdek](#). *Data Structures and Algorithms in C++*, Third Edition. Thomson Course Technology, 2005. [ISBN 0-534--0 {{isbn}}](#): Check [isbn](#) value: length ([help](#)). Chapter 4: Stacks and Queues, pp. 137–169.

[Donald Knuth](#). *The Art of Computer Programming*, Volume 1: *Fundamental Algorithms*, Third Edition. Addison-Wesley, 1997. [ISBN 0-201-89683-4](#). Section 2.2.1: Stacks, Queues, and Dequeues, pp. 238–243.

William Ford, [William Topp](#). *Data Structures with C++ and STL*, Second Edition. Prentice Hall, 2002. [ISBN 0-13-085850-1](#). Chapter 8: Queues and Priority Queues, pp. 386–390.